Manual Method 3: Internal Switch With RRAS NAT And Port Forwarding

Best For

Use this method when the VM cannot be placed directly on the LAN, but you want a more controlled Windows Server routing setup than Internet Connection Sharing.

Expected result:

LAN PC -> http://192.168.1.10:8000 -> Windows Server portproxy -> Linux VM 192.168.100.50:8000

In this method:

- Hyper-V uses an Internal switch for the VM.
- Windows Server RRAS provides NAT from the VM network to the LAN.
- Windows portproxy forwards app traffic from the Windows Server LAN IP to the VM.
- LAN users connect to the Windows Server IP or DNS name.

Requirements

- Windows Server 2012 with the Remote Access / Routing role available
- Administrator access on the Windows Server host
- A Linux VM attached to an Internal Hyper-V switch

Network Diagram

Internet
  |
Router / LAN gateway 192.168.1.1
  |
Windows Server LAN IP 192.168.1.10
  |
RRAS NAT
  |
Hyper-V Internal Switch: Form14-RRAS
  |
Linux VM private IP 192.168.100.50
  |
Docker app: 0.0.0.0:8000

Screenshot Slots

Save screenshots in:

manual_screenshots/method_3_rras_nat/

Screenshot	File Name	What To Capture
R01	R01-internal-switch.png	Hyper-V Virtual Switch Manager showing Form14-RRAS as Internal.
R02	R02-vethernet-static-ip.png	Windows adapter settings for vEthernet (Form14-RRAS) with static private IP.
R03	R03-rras-role-installed.png	Server Manager showing Remote Access/Routing role installed.
R04	R04-rras-nat-interfaces.png	RRAS console showing public and private NAT interfaces.
R05	R05-linux-route-apt.png	Linux terminal showing private IP, default route, DNS, and apt update.
R06	R06-portproxy-rule.png	Windows command output showing the portproxy rule.
R07	R07-lan-browser-windows-host.png	LAN browser showing the app through Windows Server IP or domain.

Step 1: Create The Internal Hyper-V Switch

On Windows Server 2012:

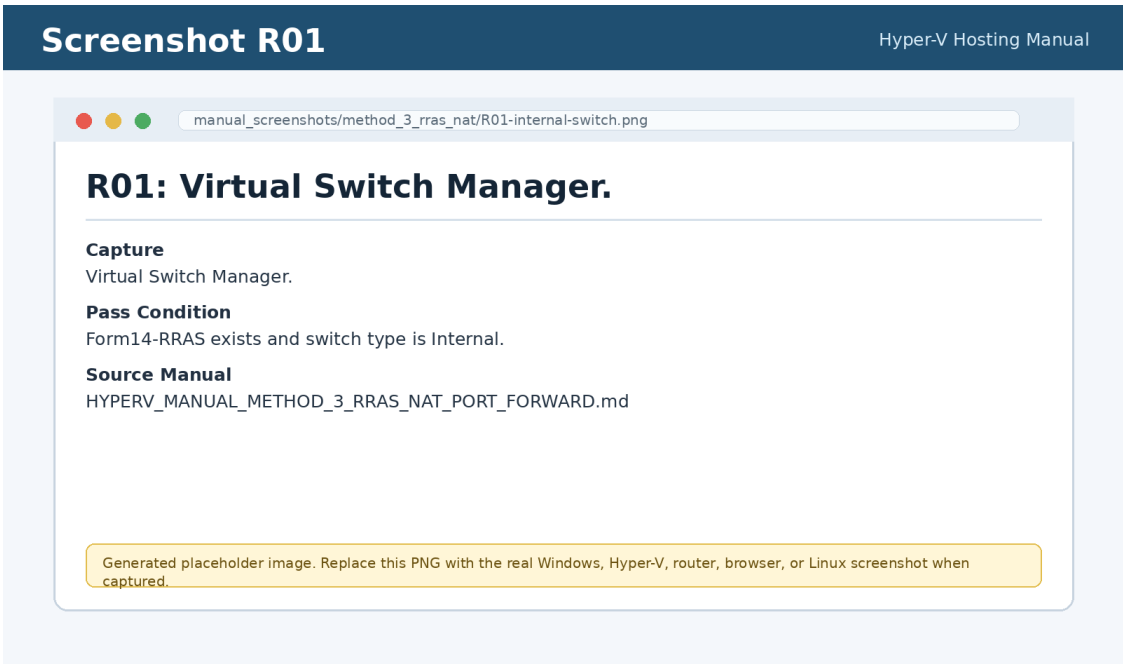
1. Open **Hyper-V Manager**.
2. Open **Virtual Switch Manager**.
3. Create a new **Internal** switch.
4. Name it:

Form14-RRAS

5. Click **Apply**.
6. Click **OK**.

Screenshot R01:

File: manual_screenshots/method_3_rras_nat/R01-internal-switch.png
Capture: Virtual Switch Manager.
Pass condition: `Form14-RRAS` exists and switch type is Internal.



{width=6.8in}

Step 2: Set The Internal Windows Adapter IP

Open **Network Connections** on Windows.

Find:

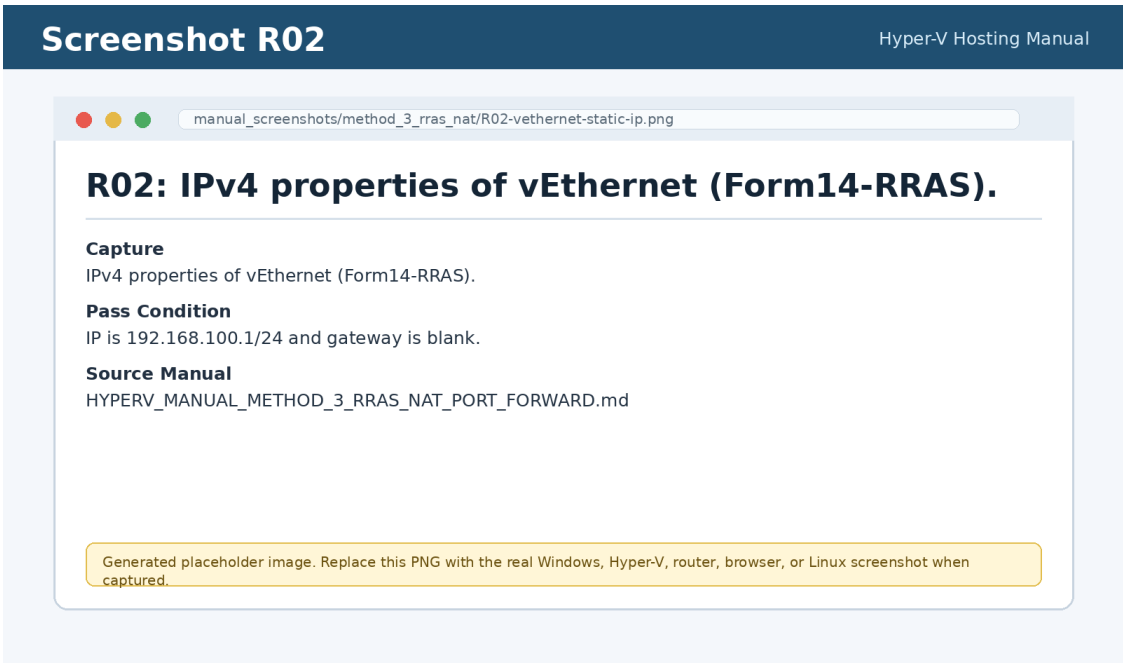
vEthernet (Form14-RRAS)

Set IPv4 manually:

IP address: 192.168.100.1
Subnet mask: 255.255.255.0
Default gateway: leave blank
DNS server: leave blank or use 127.0.0.1 if required by local policy

Screenshot R02:

File: manual_screenshots/method_3_rras_nat/R02-vethernet-static-ip.png
Capture: IPv4 properties of `vEthernet (Form14-RRAS)`.
Pass condition: IP is `192.168.100.1/24` and gateway is blank.



{width=6.8in}

Step 3: Attach The Linux VM

1. Shut down the Linux VM.
2. Open VM **Settings**.
3. Select **Network Adapter**.
4. Set **Virtual switch** to:

Form14-RRAS

5. Start the VM.

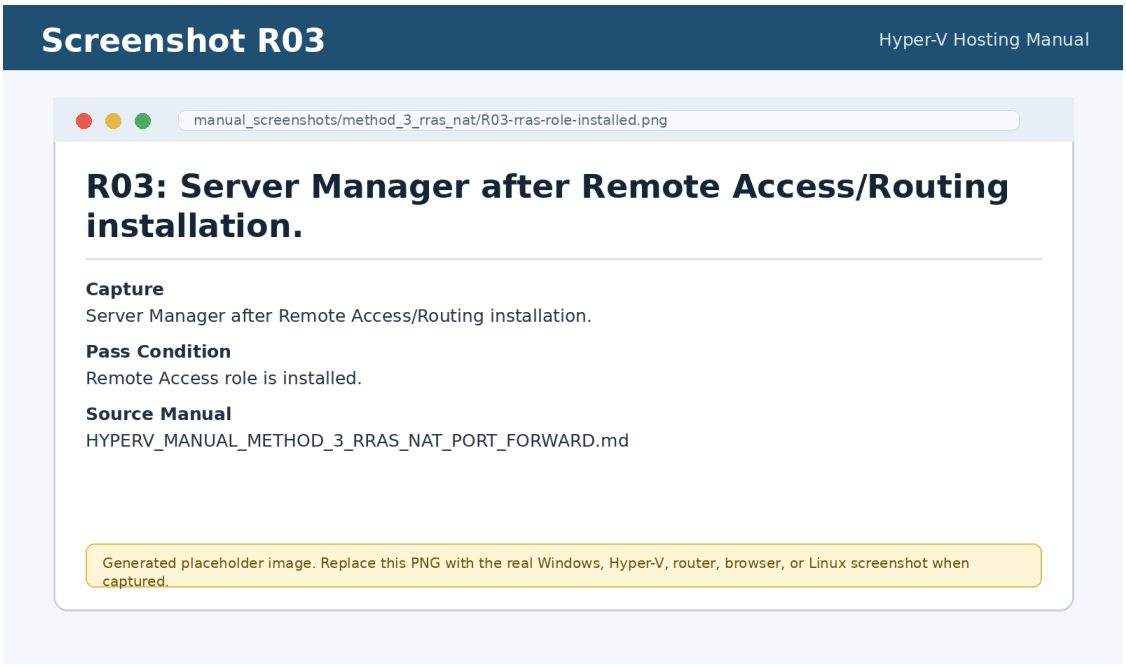
Step 4: Install And Configure RRAS NAT

On Windows Server 2012:

1. Open **Server Manager**.
2. Click **Manage**.
3. Click **Add Roles and Features**.
4. Select **Role-based or feature-based installation**.
5. Select the local server.
6. Add **Remote Access**.
7. Include **Routing** role service.
8. Complete the wizard and install.

Screenshot R03:

File: manual_screenshots/method_3_rras_nat/R03-rras-role-installed.png
Capture: Server Manager after Remote Access/Routing installation.
Pass condition: Remote Access role is installed.



{widt
h=6.8in}

Configure RRAS:

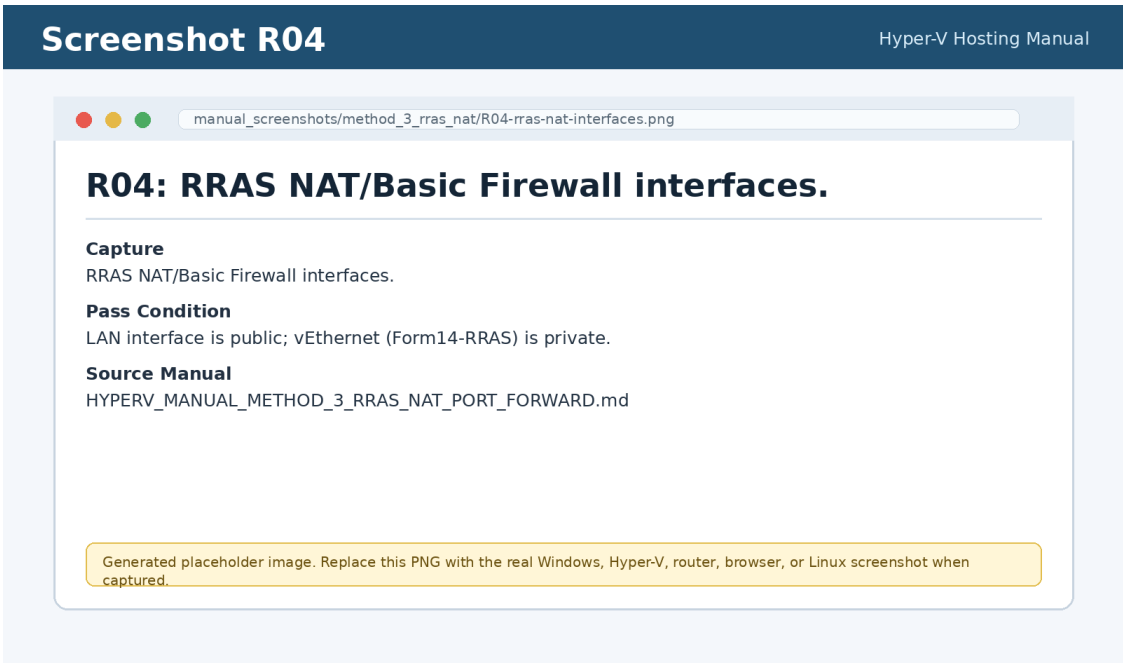
1. Open **Routing and Remote Access**.
2. Right-click the server name.
3. Click **Configure and Enable Routing and Remote Access**.
4. Select **Network address translation (NAT)**.
5. Select the public interface that has LAN/internet access.
6. Mark it as the public interface connected to the internet.
7. Add the private interface:

vEthernet (Form14-RRAS)

8. Mark it as private.

Screenshot R04:

File: manual_screenshots/method_3_rras_nat/R04-rras-nat-interfaces.png
Capture: RRAS NAT/Basic Firewall interfaces.
Pass condition: LAN interface is public; `vEthernet (Form14-RRAS)` is private.



{width=6.8in}

Step 5: Configure Linux VM Private IP

Inside Linux:

sudo nano /etc/netplan/01-netcfg.yaml

Example:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: no
      addresses:
        - 192.168.100.50/24
      routes:
        - to: default
          via: 192.168.100.1
      nameservers:
        addresses:
          - 192.168.100.1
          - 1.1.1.1
          - 8.8.8.8
```

Apply:

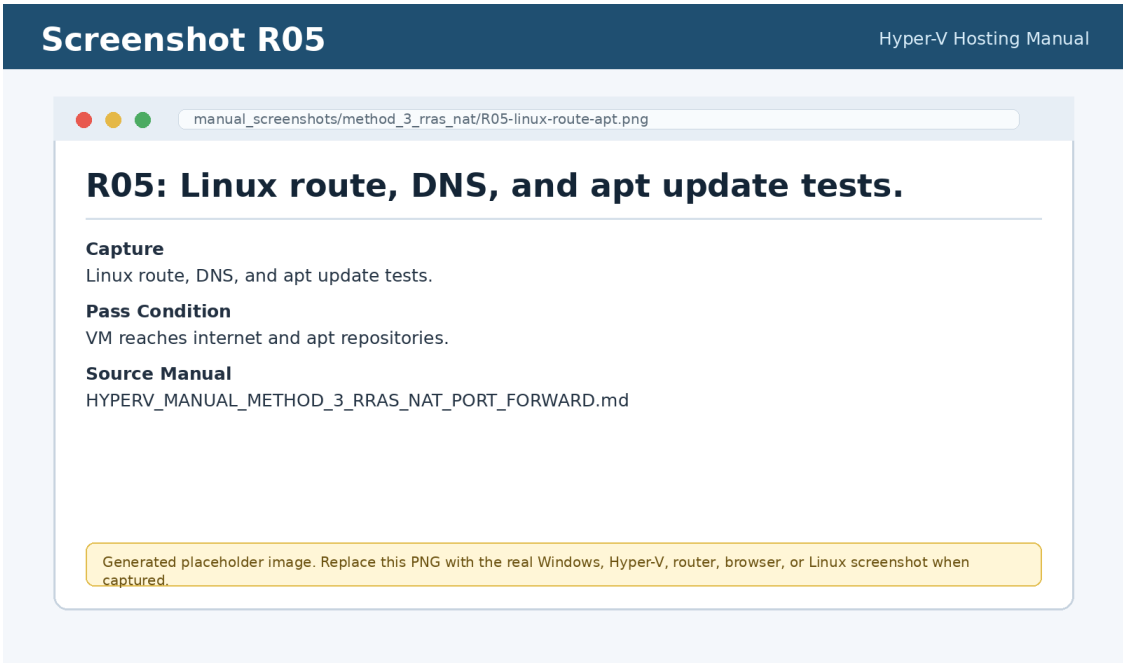
```
sudo netplan apply
```

Test:

```
ip -4 addr
ip route
ping -c 4 192.168.100.1
ping -c 4 8.8.8.8
getent hosts google.com
sudo apt update
```

Screenshot R05:

File: manual_screenshots/method_3_rras_nat/R05-linux-route-apt.png
Capture: Linux route, DNS, and apt update tests.
Pass condition: VM reaches internet and apt repositories.



{widt
h=6.8in}

Step 6: Deploy The Flask App In Docker

On Linux:

```
sudo mkdir -p /srv/form14
sudo chown -R "$USER:$USER" /srv/form14
cd /srv/form14
```

Copy the repository to /srv/form14.

Create:

nano /srv/form14/.env.production

Example:

```
HOST_PORT=8000
PORT=8000

SECRET_KEY=replace-with-a-long-random-secret
ALLOWED_HOSTS=127.0.0.1,localhost,192.168.100.50,192.168.1.10,form14.local
PUBLIC_BASE_URL=http://192.168.1.10:8000

POSTGRES_DB=form14
POSTGRES_USER=form14
POSTGRES_PASSWORD=replace-with-a-strong-db-password
INTERNAL_DATABASE_URL=postgresql://form14:replace-with-a-strong-db-
password@db:5432/form14?sslmode=disable

ADMIN_USER_EMAIL=admin@example.local
ADMIN_USER_PASSWORD=replace-with-a-strong-admin-password

CRUD_ONLY_MODE=true
OCR_FEATURES_ENABLED=false
VISUAL_ANALYSIS_ENABLED=false
SECTOR_ANALYTICS_ENABLED=false
TF_RISK_ENABLED=false
DATA_ANALYSIS_BOT_ENABLED=false
```

Install and deploy:

```
cd /srv/form14
chmod +x required.sh deploy.sh update.sh backup.sh restore.sh healthcheck.sh
./required.sh postgres
```

Verify inside Linux:

```
cd /srv/form14
docker compose --env-file .env.production -f docker-compose.prod.postgres.yml ps
curl -I http://127.0.0.1:8000
curl -I http://192.168.100.50:8000
```

Step 7: Forward Windows Host Port To The VM

On Windows Server 2012, open Command Prompt as Administrator.

Create a portproxy rule:

netsh interface portproxy add v4tov4 listenaddress=192.168.1.10 listenport=8000 connectaddress=192.168.100.50 connectport=8000

Open Windows Firewall:

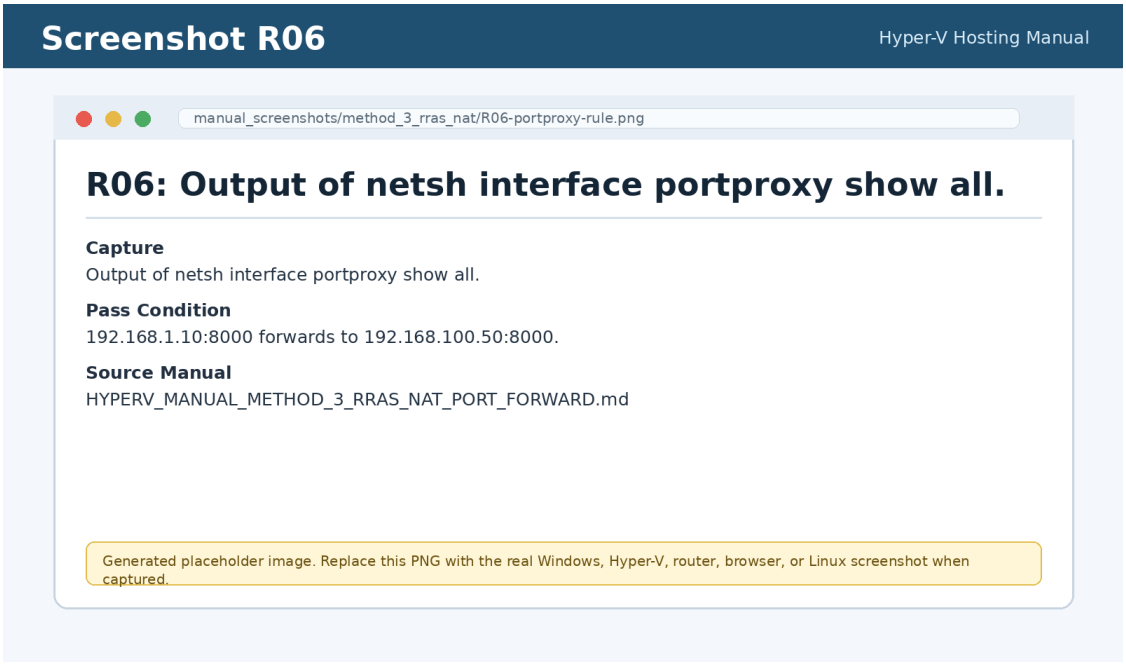
netsh advfirewall firewall add rule name="FORM14 Flask 8000" dir=in action=allow protocol=TCP localport=8000

Confirm:

netsh interface portproxy show all

Screenshot R06:

File: manual_screenshots/method_3_rras_nat/R06-portproxy-rule.png
Capture: Output of `netsh interface portproxy show all`.
Pass condition: `192.168.1.10:8000` forwards to `192.168.100.50:8000`.



{width=6.8in}

Ensure IP Helper is running:

sc query iphlpsvc
net start iphlpsvc

Step 8: Test From LAN

From another LAN computer:

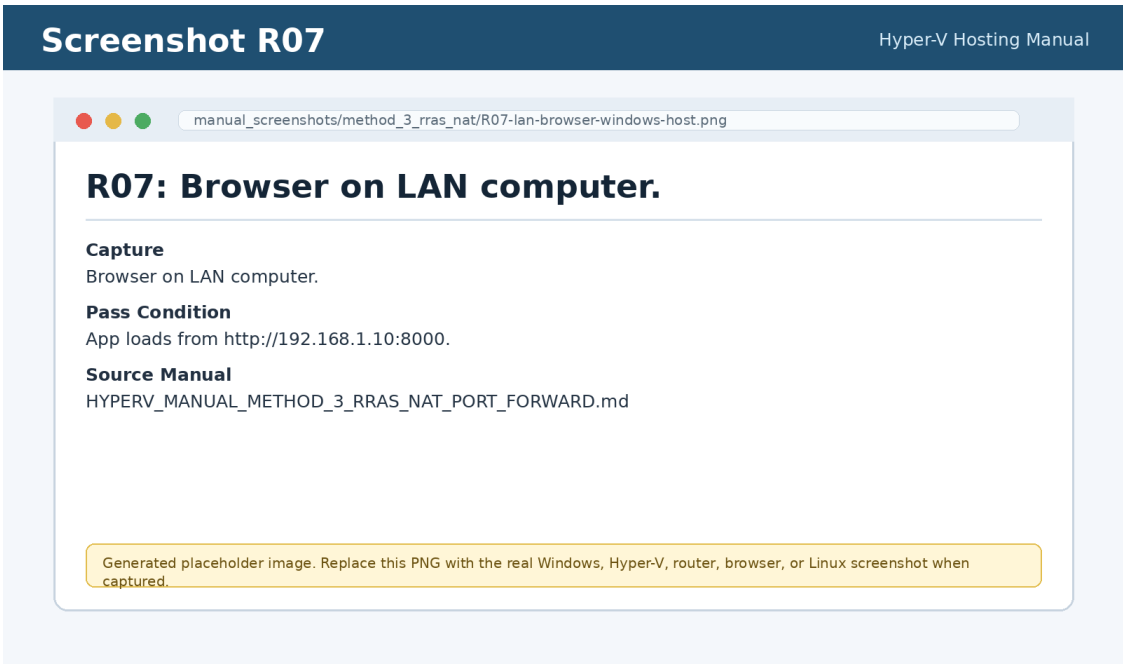
http://192.168.1.10:8000

Optional Windows test:

```
powershell -Command "$c=New-Object Net.Sockets.TcpClient;
$c.Connect('192.168.1.10',8000); $c.Close(); 'Port 8000 reachable'"
```

Screenshot R07:

File: manual_screenshots/method_3_rras_nat/R07-lan-browser-windows-host.png
Capture: Browser on LAN computer.
Pass condition: App loads from `http://192.168.1.10:8000`.



{width=6.8in}

Optional LAN Name

Point DNS or a hosts-file entry to the Windows Server IP:

192.168.1.10 form14.local

Users then open:

http://form14.local:8000

Success Criteria

This method is complete when:

- RRAS NAT is enabled.
- The VM has private IP 192.168.100.50.
- The VM can run sudo apt update.
- Docker containers are running.

- Windows portproxy forwards 192.168.1.10:8000 to the VM.
- LAN users open the app through the Windows Server IP or local DNS name.